

DATA STRUCTURE

AVL Tree

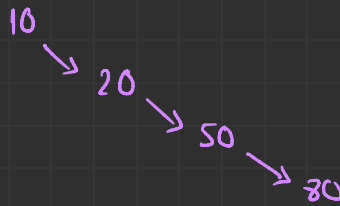
Sakhawat Adib Shuvro
210042106, SWE, IUT

AVL Tree

→ in binary tree there no restriction of order

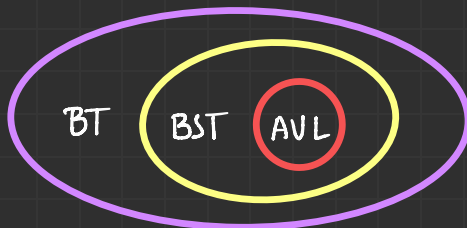
→ in binary Search Tree worst case has complexity of a list.

in a binary tree →



the data comes in sorted way that makes a list
so, height becomes (n) .

AVL Tree → Balanced Binary Search Tree



AVL Tree enforces to keep the height always

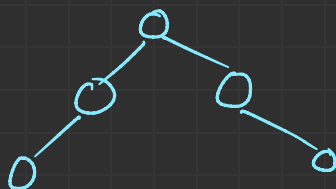
$$\log(n)$$

Balance Factor = height of Left subtree - h of RST

Balance Factor (BF) is permitted to have value,

$$-1, 0, 1$$

Suppose a binary Tree,



$$h(LST) = 2$$

$$h(RST) = 2$$

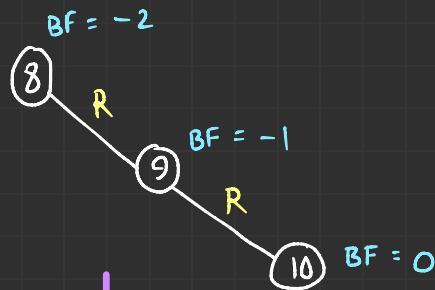
so it is an AVL

if balance factor of Binary tree keeps in range for every node then it is an AVL.

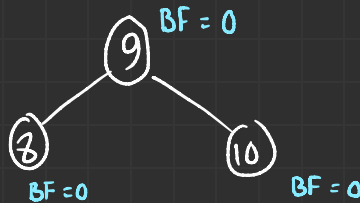
The process to draw an AVL Tree

R-R data $\rightarrow$ 8, 9, 10

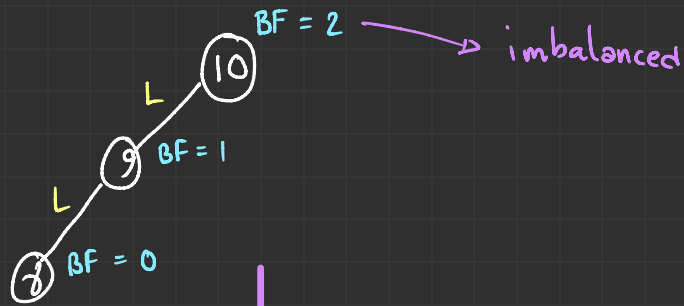
* make a binary Search Tree



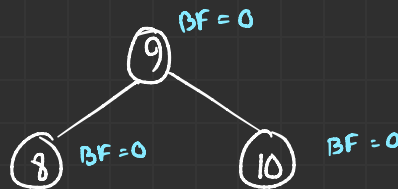
1 Anti Clock Rotation at ⑨



LL, LL Data $\rightarrow$ 10, 9, 8

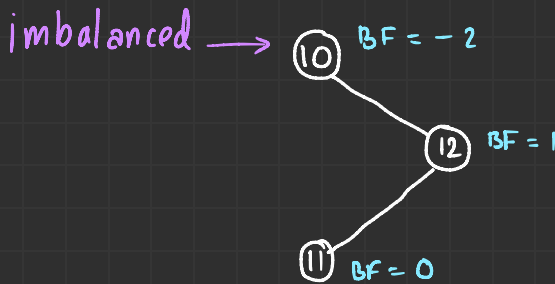


1 Clock wise Rotation at 9

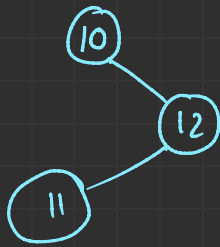


R - L

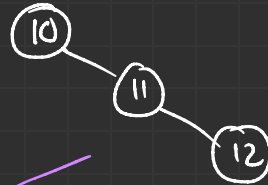
Data → 10, 12, 11



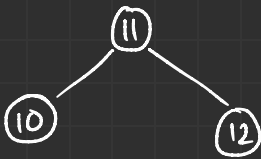
→ first convert a R-L to RR
then balance that.



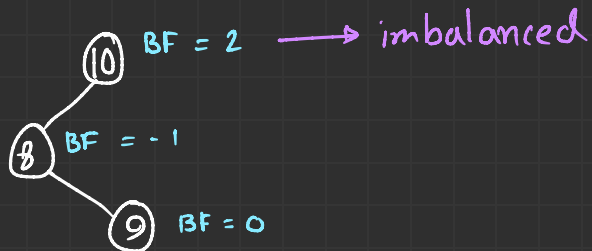
convert to RR



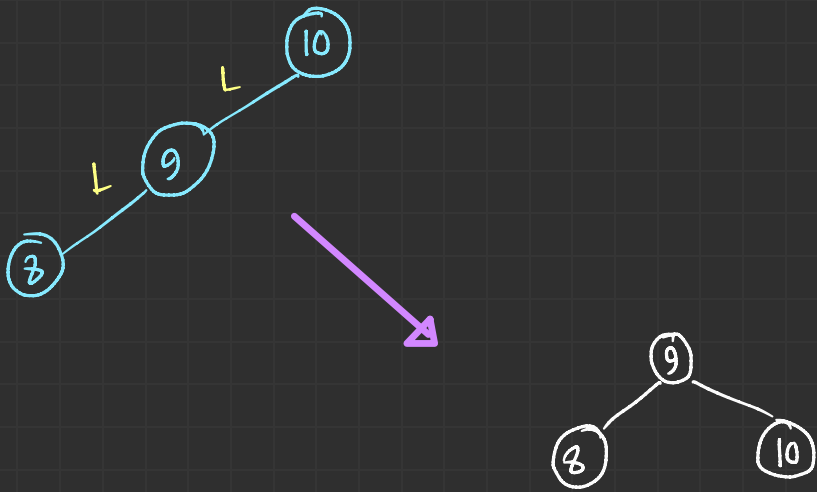
convert to balance



Data $\rightarrow$ 10 , 8 , 9

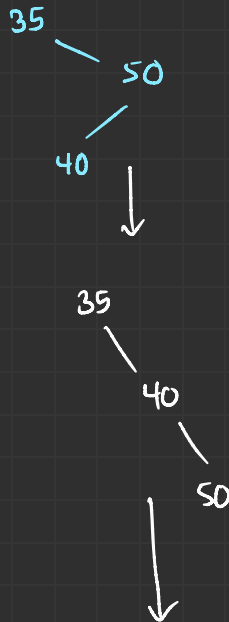


convert this to LL

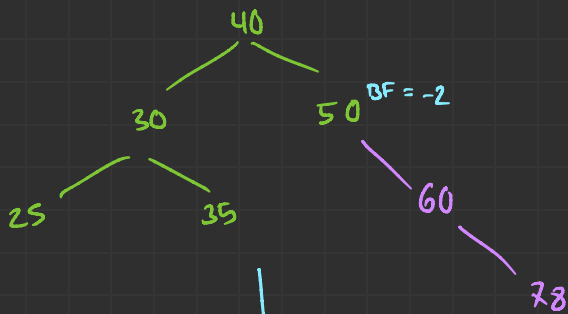
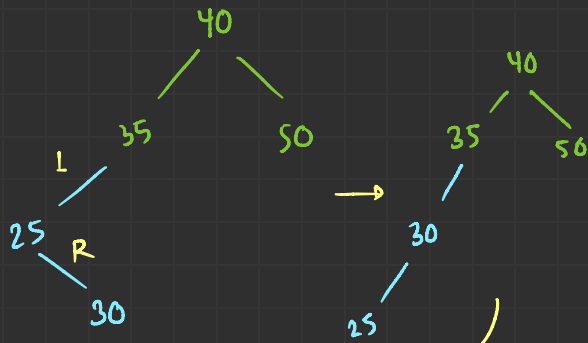


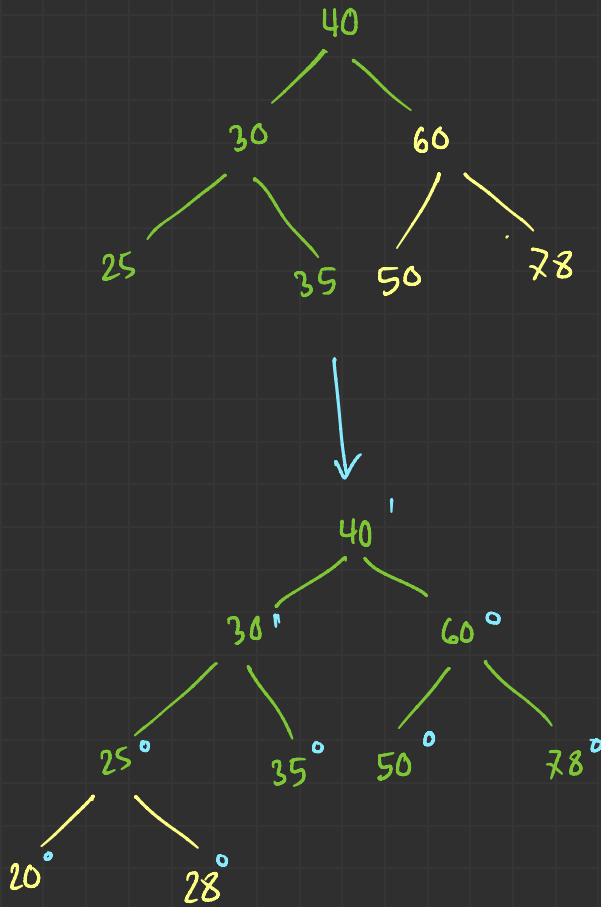
How many Rotations Are needed to Balance?

35, 50, 40, 25, 30, 60, 78, 20, 28



1. R
2. L
3. L
4. R
5. L



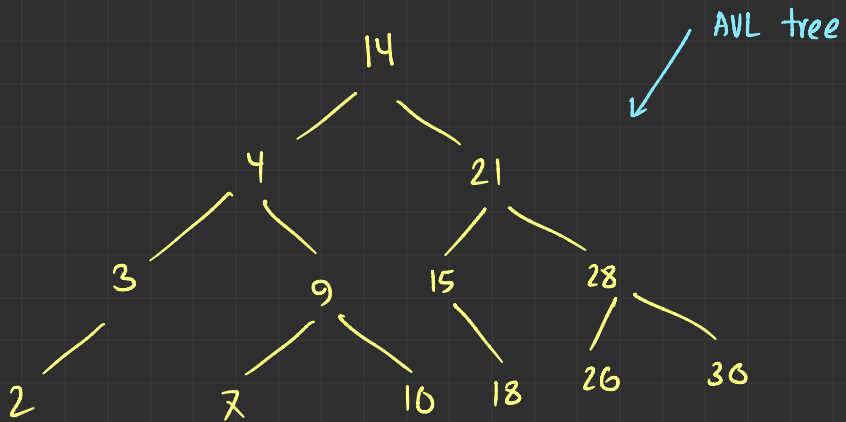


Max BF = 1

this tree is now an AVL tree.

so 3 Left Rotations and 2 Right Rotations
to get a AVL tree

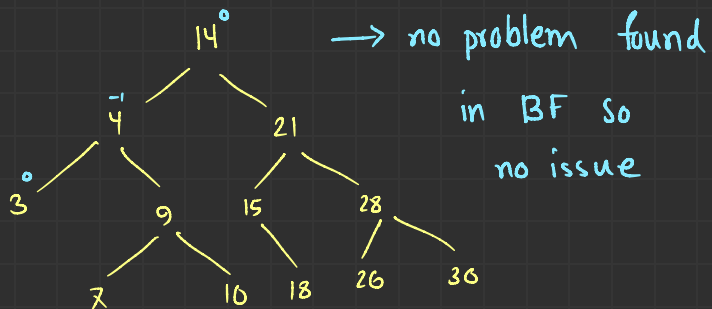
AVL Tree Deletion



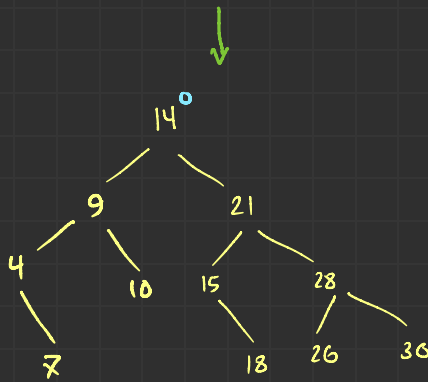
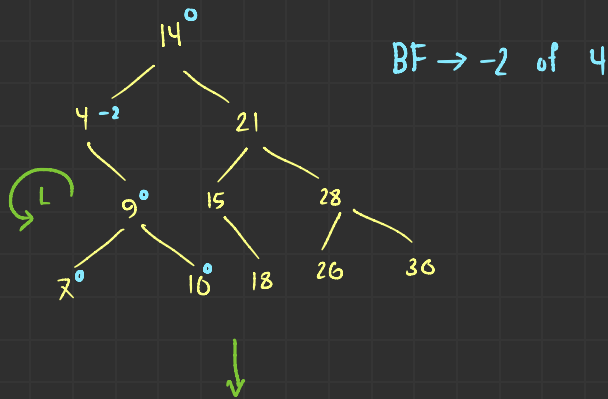
Delete following nodes in sequence →

2, 3, 10, 18, 4, 9, 14, 7, 15

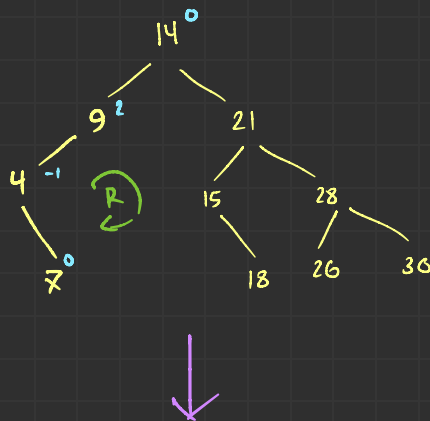
— 2 is a leaf node. Delete directly

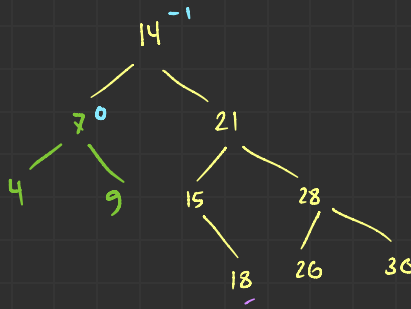


→ 3 is Also a leaf node. Delete directly.

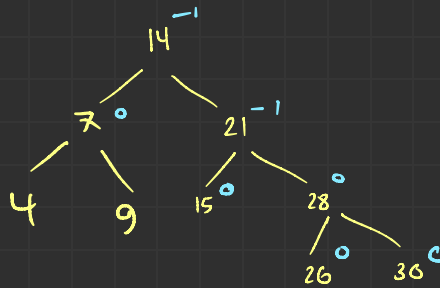


→ 10 is a leaf node. Delete directly.



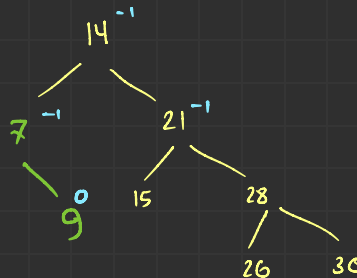


Delete 18 → 18 leaf node, Delete no issue



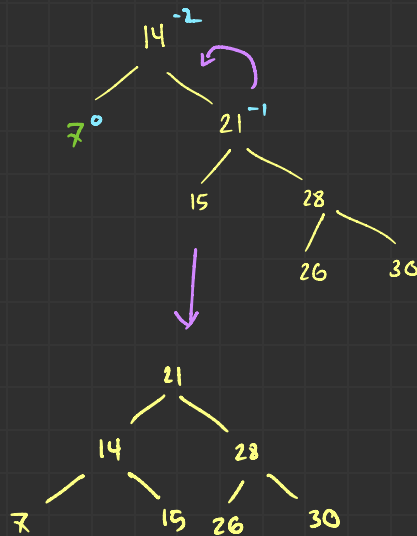
fine

Delete 4 → 4 leaf node. No issue



fine

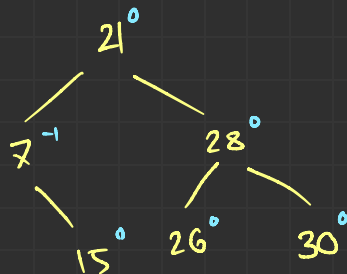
Delete 9 $\rightarrow$ leaf //



Delete 14 $\rightarrow$ internal node delete

करने Node left child से कर

Replace रहे ।



that is perfect AVL Tree